

VGEC Navigator

A shortest-path campus wayfinding tool for Vishwakarma Government Engineering College, built for the *Algorithm Analysis & Design* case study (BE05016011). It models the campus as a weighted graph and uses **Dijkstra's algorithm** to compute and animate the shortest walking route between any two locations — with dual input (search, room code, or tap-on-map) and zero backend.

 Tech

 Stack

 Backend

 Deploy

 Campus route graph with a highlighted shortest path

Table of contents

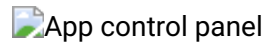
- [Features](#)
- [Demo](#)
- [How it works](#)
- [Algorithm](#)

- [Project structure](#)
 - [Run locally](#)
 - [Deploy to Vercel](#)
 - [Updating the campus data](#)
 - [Roadmap](#)
-

Features

- **Dual input modes** — search by name from dropdowns, type a room code (e.g. N207 → Block N, Floor 2, Room 07), or tap two blocks directly on the map
- **Animated shortest path** — the computed route draws itself across the campus graph in real time as an SVG stroke animation
- **Accurate campus graph** — every node is a real location (departments, hostels, GTU building, main entry); every edge is a real walking path, including the outer ring road, modeled with no fabricated shortcuts
- **Zero backend** — the entire graph is a static JSON object and Dijkstra runs client-side; deployable to Vercel with zero configuration

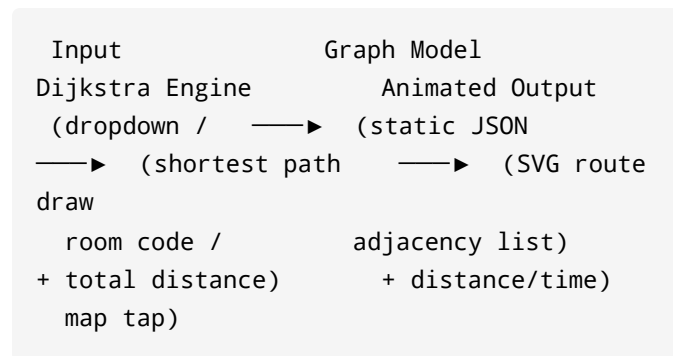
Demo



A full showcase PDF with more screenshots, the graph diagram, and the algorithm writeup is included:

[VGEC-Navigator-Showcase.pdf](#).

How it works



The campus is represented as an **undirected, weighted graph** — nodes are real locations, edges are real walking paths weighted by distance in meters. There are no synthetic "junction" nodes: the outer ring road is modeled as direct edges between the buildings that physically sit along it, so a route between two non-adjacent points (e.g. Block D → GTU) correctly walks the real path distance instead of taking an invented shortcut.

Algorithm

Dijkstra's algorithm was chosen because every edge weight is a non-negative physical distance — exactly its required precondition. It guarantees the optimal path and is simpler to implement than Bellman-Ford or Floyd-Warshall, neither of which is needed for a single source-destination query.

```
function dijkstra(source, target) {  
    // dist[] tracks shortest known distance  
    // to each node  
    // prev[] tracks the path back to source,  
    // for reconstruction  
    initialize dist[source] = 0, all others =  
    Infinity  
  
    while priority queue is not empty:  
        u = node with minimum dist[] not yet  
        visited  
        if u === target: break          // early  
        exit once settled  
        for each neighbor v of u:  
            if dist[u] + weight(u,v) < dist[v]:  
                dist[v] = dist[u] + weight(u,v)  
                prev[v] = u  
  
    return reconstructPath(prev, target),
```

```
dist[target]
}
```

Complexity

|||

|---|---|

| Time | $O((V + E) \log V)$ with a binary-heap priority queue |

| Space | $O(V + E)$ for the adjacency list, $O(V)$ for distance/parent tables |

| vs. brute force | $O(V!)$ worst case for path enumeration – Dijkstra is dramatically faster even at this small scale |

Project structure

```
vgec-navigator/
├── index.html           # the
entire app – graph data, Dijkstra, UI, all
in one file
├── README.md
├── VGEC-Navigator-Showcase.pdf  #
portfolio-style writeup with diagrams
└── images/
    ├── graph_diagram.png
    └── ui_panel.png
```

Run locally

No build step, no server required — just open `index.html` in a browser.

Or serve it locally:

```
npx serve .
```

Deploy to Vercel

1. Push this folder to a GitHub repo
2. Go to vercel.com/new and import the repo
3. Framework preset: **Other** (static site) — no build command needed
4. Deploy → you get a live public URL (great for a QR code at the campus gate)

Every push to the repo auto-redeploys.

Updating the campus data

Everything about the map lives in two blocks inside `index.html`: `const NODES = {...}` and `const EDGES = [...]`.

- **Move a building** — change its `x / y` in `NODES`
- **Add/remove a path** — add or delete a line in `EDGES`, format `["from","to",distanceInMeters]`. Both IDs must exist in `NODES`.

- **Force indirect routing** (e.g. no shortcut between two points) – make sure there's no direct edge between them; both should only connect through a shared node on the real path between them.
- Save, reopen `index.html` in a browser, tap the two blocks you changed, and hit "Find shortest route" to verify.

Note on distances: edge weights are scaled from the campus site-plan layout, not GPS/pedometer-measured. Verify with a real walk-through or Google Maps' measure tool before treating them as authoritative.

Roadmap

- ☐ Replace scaled-estimate distances with real measured values
 - ☐ Real GPS coordinates + "use my current location"
 - ☐ Indoor/multi-floor navigation for room-level routing
 - ☐ Live path-blockage toggle for real-time closures
-

Built as part of the Algorithm Analysis & Design case study, Dept. of Information Technology, VGEC.